

Operating System: Scheduling Algo

Dr. Jit Mukherjee



Motivation

- In a single processor system only one process run at a time. Others have to wait.
- Multiprogramming - several process are kept in memory at one time. When one process has to wait, the OS takes the CPU away from that process and gives the CPU to another process.
- Process execution consists of a cycle of CPU execution and I/O wait
- Process execution is a combination of CPU burst and IO burst.
- A CPU bursts when it is executing instructions; an I/O system bursts when it services requests to fetch information.
- Whenever the CPU is idle, the OS must select a process from the ready queue using short-term scheduler.
 - The records in the queue are generally PCB of the process

When we do Scheduling

1. When a process switches from the running state to the waiting state
 - a. As a result of an I/O request or an invocation of wait for the termination of a child process.
2. When a process switches from the running state to the ready state
 - a. Interrupts
3. When a process switches from the waiting state to ready state
 - a. After completing an I/O operation
4. When a process terminates.

For 1 and 4 there is no choice in term of scheduling. A new process, if one is in ready queue, has to be chosen - **Non Preemptive or Cooperative**

There is a choice for situation 2 and 3 - **Preemptive**

Non Preemptive and Preemptive Scheduling

- In non preemptive, the process keeps the cpu until it releases the CPU
 - By terminating
 - By switching to waiting state
 - Windows 3.x
- It can be used in certain platforms which does not requires special hardware like a timer.
- Preemptive - Windows 95 onwards, Mac OS
 - Problem while accessing shared data.
 - Two process share a data. While one process is updating the data, it is preempted and second process tries to read the data.
 - Kernel call changing important kernel data, such as device driver, while it is preempted.
 - Kernel will not preempt while kernel data structure is in inconsistent state - Unix
- Dispatcher - Gives control of the CPU to the process selected.
 - Dispatcher latency

Scheduling Criteria

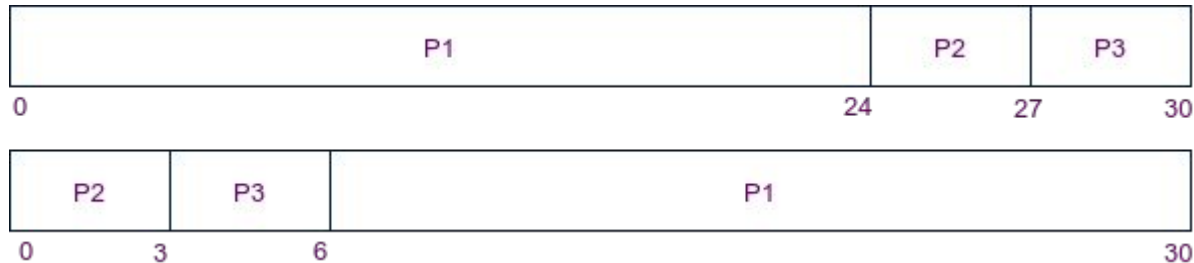
- CPU Utilization - Keep the CPU as busy as possible - Maximise
- Throughput - Number of processes completed per unit time. - Maximise
 - Long process - may be one process per hour, Short process - ten processes per second
- Turnaround Time - How long it takes to execute the process. - Minimise
 - Time of completion - Time of submission
 - Sum of all waiting and execution
- Waiting Time - Sum of the periods waiting in the ready queue. - Minimise
 - Not I/O waiting
- Response Time - In interactive system turnaround time may not be best criteria
 - A process can produce some output early and can continue computing new results while previous results are output to the user.
 - The time from the submission of a request until first response is produced. - Minimise
 - The time to start responding not the time it takes to output the response
 - Minimize the variance of response time rather than average response time

Scheduling Algorithm

- First-Come, First-Served (FCFS) Scheduling
- Shortest-Job-First (SJF) Scheduling
- Priority Scheduling
- Shortest Remaining Time
 - This is preemptive SJF
- Round Robin(RR) Scheduling
- Multiple-Level Queues Scheduling

First Come First Serve (FCFS)

- Simplest Algorithm. Can be managed by FIFO queue.
- Process enters ready queue -> PCB is linked onto the tail.
- When CPU is free, process at the head is allocated.
- It is nonpreemptive, i.e cooperative.
 - Troublesome
- Process - P1, P2, P3
- Burst Time - 24, 3, 3



- Gantt Chart
- First average waiting time
 - $(0+24+27)/3 = 17$ milliseconds
- Second Average Waiting Time
 - $(6+0+3)/3 = 3$ milliseconds
- Problem with CPU and I/O bound Process

Shortest Job First

- Associate each process with length of the process's CPU next burst
- When CPU is available it assigns the process which has lowest next CPU burst
- If the next CPU burst is same, FCFS is used among them
- Shortest Next CPU Burst Algorithm - Scheduling depends on the length of next CPU burst rather than total length.
- This is a non-preemptive, preemptive scheduling algorithm.
- Best approach to minimize waiting time. Provably optimal.
- Easy to implement in Batch systems where required CPU time is known in advance.
- Difficult to implement in interactive systems where required CPU time is not known.
- The processor should know in advance how much time process will take.

Shortest Job First

- A new process arrives at ready queue. The next CPU burst of the new process is lower than what is left of the currently executing process.

Process	Arrival Time	Execute Time	Service Time
P0	0	5	3
P1	1	3	0
P2	2	8	16
P3	3	6	8

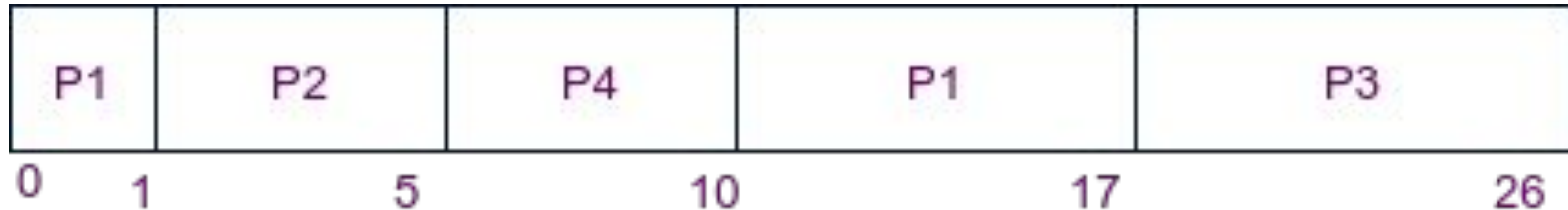


Process	Waiting Time
P0	$0 - 0 = 0$
P1	$5 - 1 = 4$
P2	$14 - 2 = 12$
P3	$8 - 3 = 5$

Shortest Remaining Time First

- Preemptive SJF
- Average waiting Time ->
 $((10-1)+(1-1)+(17-2)+(5-3))/4 = 6.5$ millisecond
- In SJF it is 7.75 miliseconds

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5



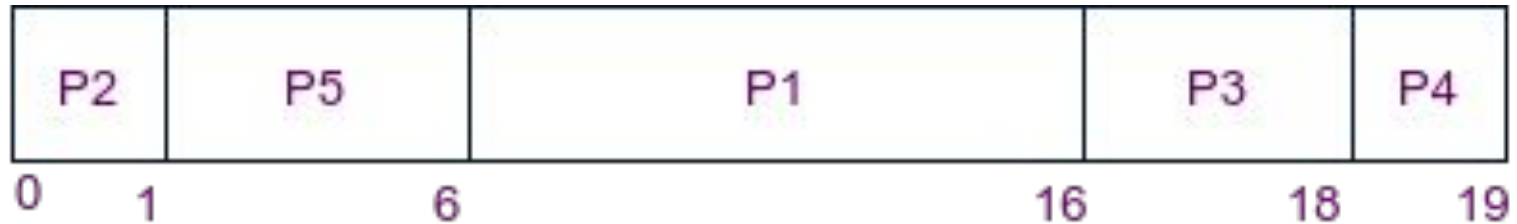
Priority Scheduling

- SJF is a priority scheduling where priority is shortest next CPU burst
- FCFS - equal priority
- In some cases 0 mean high priority, other 0 means lowest priority.
 - General - > Low number is high priority
- Each process is assigned a priority. Process with highest priority is to be executed first and so on.
- Processes with same priority are executed on first come first served basis.
- Priority can be decided based on memory requirements, time requirements or any other resource requirement - Internally Defined
- External Priority - Fund, Importance, User, Sponsorship
- Can be preemptive or non preemptive
- When a process arrives in ready queue, it's priority is compared.
 - Preemptive
 - Non Preemptive

Starvation and Aging

- Ready to run but waiting for CPU
- Heavily loaded system
 - Run when system is lightly loaded
 - Computer will crash, all the low priority job is lost
- Aging - Gradually increase the priority over time

Process	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2



Round Robin

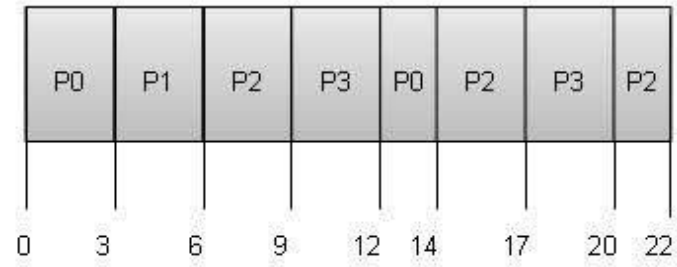
- Circular Queue. New Process will put on tail
- Round Robin is the preemptive process scheduling algorithm.
- Each process is provided a fix time to execute, it is called a quantum.
- Once a process is executed for a given time period, it is preempted and other process executes for a given time period.
- Context switching is used to save states of preempted processes.
- If there is n processes in the ready queue and time quantum in q .
 - Each Process gets $1/n$ CPU time in chunks of at most q time units
 - Each Process must wait no longer than $(n-1)*q$ time units until it's next quantum.

Round Robin

- Time quantum needs to be larger than context switch time. Otherwise CPU will be busy doing context switching mostly.
 - Modern system time quantum 10-100 milliseconds, Context Switching - 10 microseconds
- However it should not be too large. Then it becomes FCFS
- 80% of the CPU burst should be shorter than the time quantum

Average Wait Time: $(9+2+12+11) / 4 = 8.5$

Quantum = 3



Process	Wait Time : Service Time - Arrival Time
P0	$(0 - 0) + (12 - 3) = 9$
P1	$(3 - 1) = 2$
P2	$(6 - 2) + (14 - 9) + (20 - 17) = 12$
P3	$(9 - 3) + (17 - 12) = 11$